LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 3 & 4 – Function & Procedure

NAMA

NIM

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pembuatan sebuah program, kita seringkali menemui bagian kode yang harus digunakan berulang kali. Agar kode tidak menumpuk dan lebih mudah dikelola, kita menggunakan konsep fungsi (function) dan prosedur (procedure). Keduanya merupakan sekumpulan instruksi yang dikelompokkan menjadi satu blok untuk menjalankan tugas tertentu. Fungsi biasanya digunakan jika kita membutuhkan hasil perhitungan atau nilai balik (return value) untuk diproses lebih lanjut. Sedangkan prosedur lebih sering digunakan untuk menjalankan serangkaian aksi, seperti menampilkan teks ke layar, tanpa harus mengembalikan nilai ke bagian utama program.

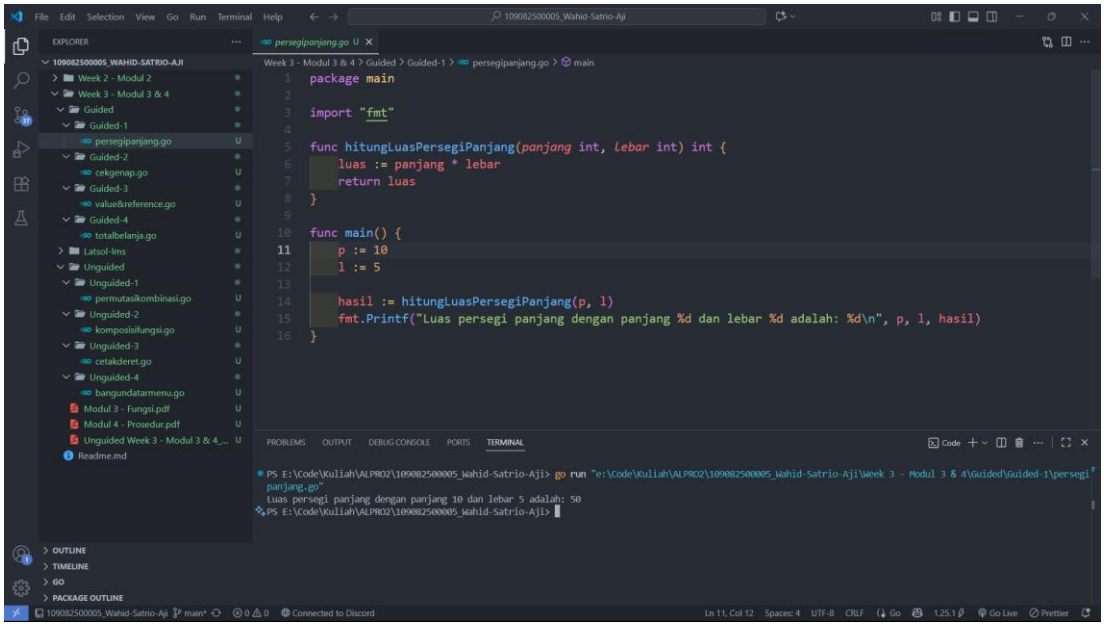
Bahasa Go memiliki fleksibilitas dalam menangani pengiriman data melalui parameter, yaitu dengan metode **pass by value** dan **pass by reference**. Pada *pass by value*, program hanya mengirimkan salinan nilainya saja, sehingga variabel asli tidak akan berubah meskipun diubah di dalam fungsi. Sebaliknya, *pass by reference* menggunakan **pointer** untuk mengirimkan alamat memori variabel asli. Dengan cara ini, perubahan yang dilakukan di dalam subprogram akan langsung mengubah nilai pada variabel aslinya. Penggunaan fungsi dan prosedur ini sangat membantu dalam membagi masalah besar menjadi bagian-bagian kecil yang lebih sederhana dan terstruktur.

B. Guided

1. Menghitung luas persegi panjang

```
package main

import "fmt"

func hitungLuasPersegiPanjang(panjang int, lebar int)
int {
    luas := panjang * lebar
    return luas
}

func main() {
    p := 10
    l := 5

    hasil := hitungLuasPersegiPanjang(p, l)
    fmt.Printf("Luas persegi panjang dengan panjang %d
dan lebar %d adalah: %d\n", p, l, hasil)
}
```

Output :

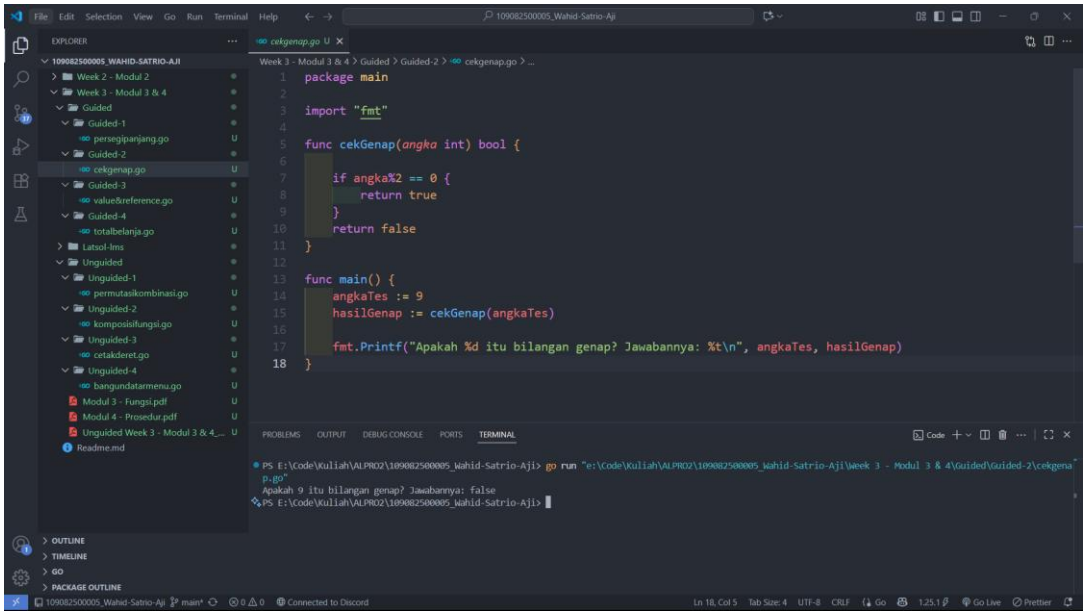


Penjelasan :
Program ini dibuat untuk menghitung luas persegi panjang menggunakan sebuah fungsi bernama hitungLuasPersegiPanjang. Fungsi ini dirancang untuk menerima dua parameter masukan, yaitu panjang dan lebar, kemudian mengalikan keduanya dan mengembalikan hasil perkalian tersebut sebagai nilai balik (return value). Di dalam program utama (main), kita menentukan nilai panjang sebesar 10 dan lebar sebesar 5, lalu memanggil fungsi tersebut untuk mendapatkan hasilnya dan menampilkannya ke layar.

2. Cek bilangan genap

```
package main

import "fmt"

func cekGenap(angka int) bool {

      if angka%2 == 0 {
            return true
      }
      return false
}

func main() {
      angkaTes := 9
      hasilGenap := cekGenap(angkaTes)

      fmt.Printf("Apakah %d itu bilangan genap? Jawabannya:
%t\n", angkaTes, hasilGenap)
}
```

Output :



Penjelasan :

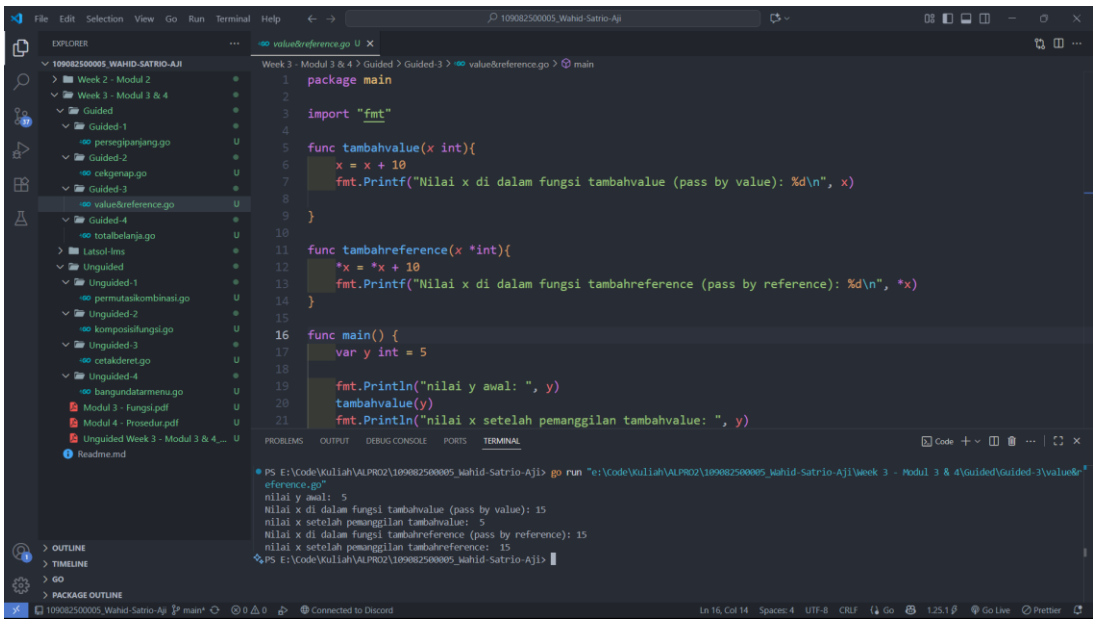
Pada program ini, kita menggunakan fungsi cekGenap yang mengembalikan tipe data boolean (true atau false). Cara kerjanya adalah dengan memeriksa sisa bagi angka yang dimasukkan terhadap angka 2 menggunakan operator modulo (%). Jika sisa baginya adalah nol, maka fungsi memberikan jawaban true yang berarti angka tersebut genap. Contoh ini menunjukkan bagaimana fungsi bisa digunakan untuk melakukan validasi atau pengecekan kondisi secara praktis.

3. Pass by value dan pas by reference

```
package main

import "fmt"

func tambahvalue(x int){
    x = x + 10
    fmt.Printf("Nilai x di dalam fungsi tambahvalue (pass by
value): %d\n", x)

}

func tambahreference(x *int){
    *x = *x + 10
    fmt.Printf("Nilai x di dalam fungsi tambahreference
(pass by reference): %d\n", *x)
```

```
}

func main() {
    var y int = 5

    fmt.Println("nilai y awal: ", y)
    tambahvalue(y)
    fmt.Println("nilai x setelah pemanggilan tambahvalue: ",
y)
    tambahreference(&y)
    fmt.Println("nilai x setelah pemanggilan
tambahreference: ", y)
}
```

Output :



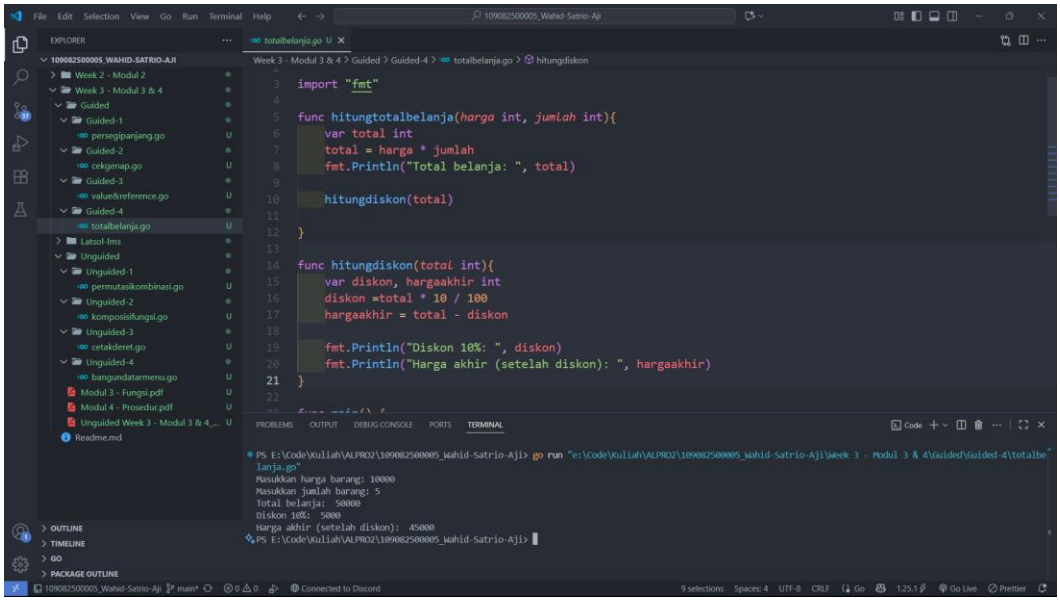
Penjelasan :

Kode ini merupakan eksperimen untuk melihat perbedaan cara kerja pengiriman parameter dalam bahasa Go. Terdapat dua fungsi berbeda: tambahvalue dan tambahreference. Fungsi tambahvalue hanya menerima salinan data saja, sehingga perubahan nilai di dalam fungsi tidak memengaruhi variabel asli di program utama. Sebaliknya, fungsi tambahreference menggunakan pointer (tanda * dan &) untuk mengakses alamat memori variabel asli. Hasilnya, nilai variabel di program utama benar-benar berubah secara permanen setelah fungsi tersebut dijalankan.

4. Hitung total belanja

```
package main

import "fmt"

func hitungtotalbelanja(harga int, jumlah int){
    var total int
    total = harga * jumlah
    fmt.Println("Total belanja: ", total)

    hitungdiskon(total)

}

func hitungdiskon(total int){
    var diskon, hargaakhir int
    diskon =total * 10 / 100
    hargaakhir = total - diskon

    fmt.Println("Diskon 10%: ", diskon)
    fmt.Println("Harga akhir (setelah diskon): ",
hargaakhir)
}

func main() {
    var harga, jumlah int

    fmt.Print("Masukkan harga barang: ")
    fmt.Scan(&harga)
    fmt.Print("Masukkan jumlah barang: ")
    fmt.Scan(&jumlah)

    hitungtotalbelanja(harga, jumlah)
}
```

Output :



Penjelasan :

Program ini mensimulasikan sistem kasir sederhana menggunakan prosedur yang saling terhubung. Di sini, prosedur hitungtotalbelanja bertugas menghitung total harga awal dari harga barang dikali jumlahnya. Menariknya, di dalam prosedur tersebut terdapat pemanggilan prosedur lain yaitu hitungdiskon. Prosedur kedua ini berfungsi menghitung potongan harga sebesar 10% dan langsung menampilkan rincian harga akhir yang harus dibayar. Alur ini menunjukkan bagaimana prosedur bisa digunakan untuk membagi tugas-tugas spesifik dalam satu rangkaian proses.

C. Unguided

1. Soal Unguided 1

1. Minggu ini, mahasiswa Fakultas Informatika mendapatkan tugas dari mata kuliah matematika diskrit untuk mempelajari kombinasi dan permutasi. Jonas salah seorang mahasiswa, iseng untuk mengimplementasikannya ke dalam suatu program. Oleh karena itu buatlah program yang dapat menghitung kombinasi & permutasi.

   Masukan terdiri dari empat buah bilangan asli a, b, c, d dengan syarat $a \geq c$ dan $b \geq d$

   Keluaran terdiri dari empat baris. Baris pertama adalah hasil permutasi a terhadap c, baris kedua adalah hasil kombinasi a terhadap c, baris ketiga adalah hasil permutasi b terhadap d, dan baris keempat adalah hasil kombinasi b terhadap d.

   Catatan Permutasi (P) dan kombinasi (C) dari n terhadap r ($n \geq r$) dapat dihitung dengan menggunakan persamaan berikut!

   $$P(n,r) = \frac{n!}{(n-r)!}, \text{ sedangkan } C(n,r) = \frac{n!}{r!(n-r)!}$$

   Selesaikan program tersebut dengan memanfaatkan subprogram yang diberikan dibawah ini :

```
Function factorial (n : integer) → integer
{I.S. terdefinisi bilangan bulat positif n
 F.S. mengembalikan nilai factorial dari n}

Function permutasi(n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n permutasi r}

Function kombinasi (n, r : integer) → integer
{I.S. terdefinisi bilangan bulat positif n dan r, dengan n >= r
 F.S. mengembalikan hasil n kombinasi r}
```

   Hitunglah permutasi dan kombinasi apabila :

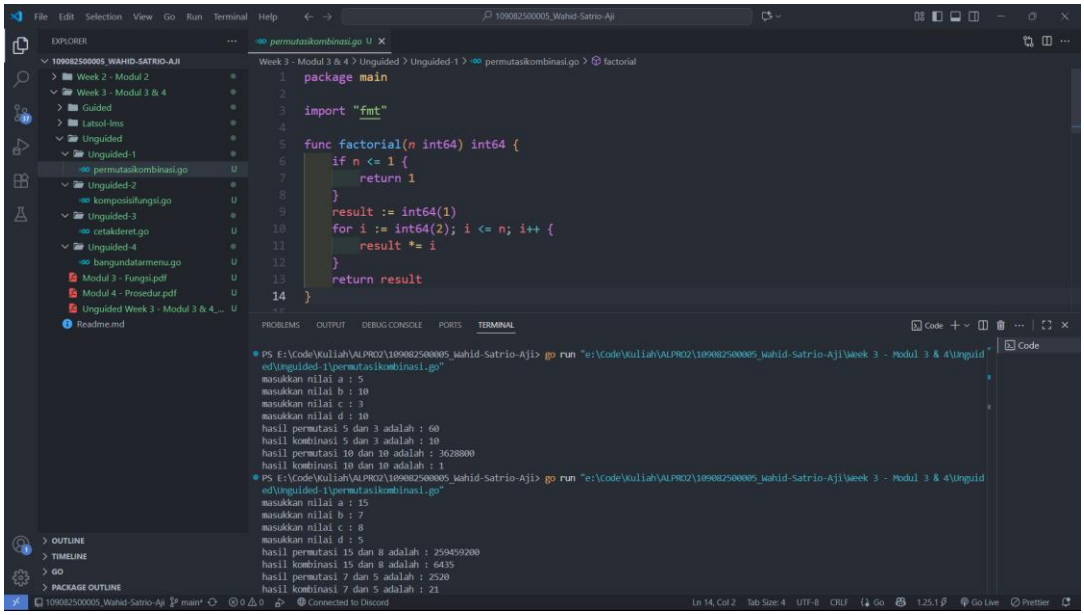
   1) Nilai a = 5, Nilai b = 10, Nilai c = 3, Nilai d = 10
   2) Nilai a = 15, Nilai b = 7, Nilai c = 8, Nilai d = 5

Jawaban :

```
package main

import "fmt"

func factorial(n int64) int64 {
	if n <= 1 {
		return 1
	}
	result := int64(1)
	for i := int64(2); i <= n; i++ {
		result *= i
```

```
        }
        return result
    }

    func permutasi(n, r int64) int64 {
        return factorial(n) / factorial(n-r)
    }

    func kombinasi(n, r int64) int64 {
        return permutasi(n, r) / factorial(r)
    }

    func main() {
        var a, b, c, d int64

        fmt.Print("masukkan nilai a : ")
        fmt.Scan(&a)
        fmt.Print("masukkan nilai b : ")
        fmt.Scan(&b)
        fmt.Print("masukkan nilai c : ")
        fmt.Scan(&c)
        fmt.Print("masukkan nilai d : ")
        fmt.Scan(&d)

        fmt.Printf("hasil permutasi %d dan %d adalah :
    %d\n", a, c, permutasi(a, c))
        fmt.Printf("hasil kombinasi %d dan %d adalah :
    %d\n", a, c, kombinasi(a, c))
        fmt.Printf("hasil permutasi %d dan %d adalah :
    %d\n", b, d, permutasi(b, d))
        fmt.Printf("hasil kombinasi %d dan %d adalah :
    %d\n", b, d, kombinasi(b, d))
}
```

Output :



Penjelasan :

Program ini dibuat untuk menghitung nilai permutasi dan kombinasi dari dua pasang bilangan yang dimasukkan oleh pengguna. Agar kode lebih efisien dan terorganisir, logika perhitungan dibagi ke dalam beberapa fungsi. Fungsi pertama adalah factorial yang menghitung nilai faktorial secara iteratif menggunakan perulangan. Selanjutnya, terdapat fungsi permutasi yang menggunakan hasil faktorial untuk menghitung susunan objek, serta fungsi kombinasi yang memanfaatkan fungsi permutasi dan faktorial untuk menghitung pengelompokan objek. Di bagian main, program akan meminta empat input sekaligus (a, b, c, d) dan menampilkan hasil perhitungan permutasi serta kombinasi untuk pasangan (a, c) dan (b, d) secara berurutan.

2. Soal Unguided 2

2. Diberikan tiga buah fungsi matematika yaitu :
   - $f(x) = x^2$
   - $g(x) = x - 2$
   - $h(x) = x + 1$

   Fungsi komposisi $(f\ o\ g\ o\ h)(x)$ artinya $f(g(h(x)))$. Tuliskan $f(x)$, $g(x)$, dan $h(x)$ dalam bentuk function.

   Masukan terdiri dari sebuah bilangan bulat a, b, c

   Keluaran terdiri dari 3 baris, baris pertama adalah $(f\ o\ g\ o\ h)(a)$, baris kedua adalah $(g\ o\ h\ o\ f)(b)$, dan baris ketiga adalah $(h\ o\ f\ o\ g)(c)$.

   hitunglah apabila masukan :

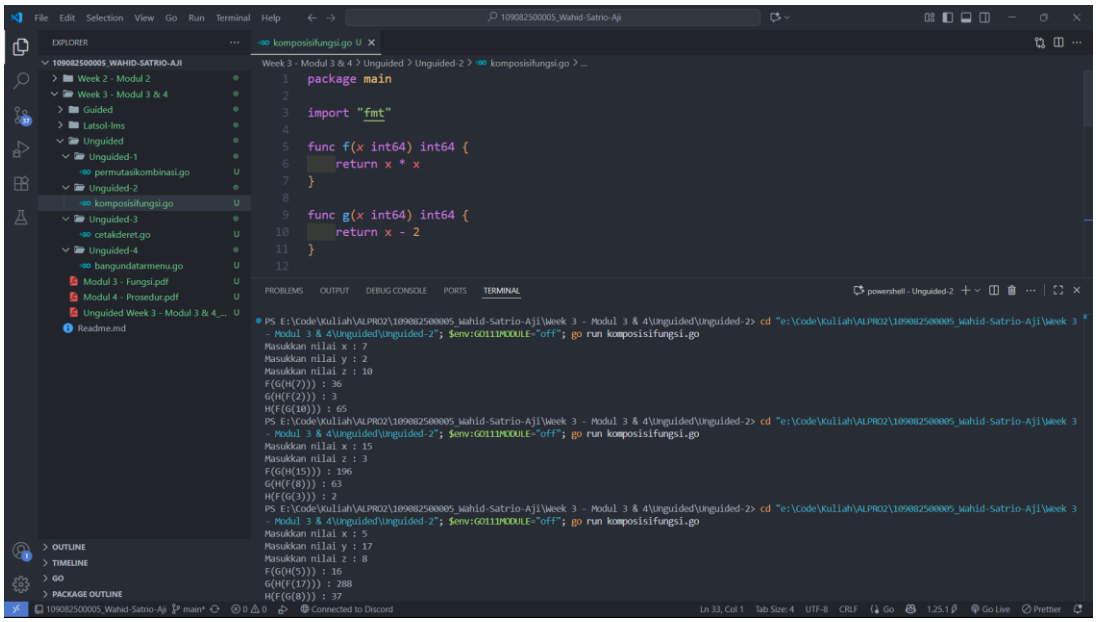
   1) Nilai a = 7, nilai b = 2, nilai c = 10
   2) Nilai a = 15, nilai b = 8, nilai c = 3
   3) Nilai a = 5, nilai b = 17, nilai c = 8

Jawaban :

```
package main

import "fmt"

func f(x int64) int64 {
      return x * x
}

func g(x int64) int64 {
      return x - 2
}

func h(x int64) int64 {
      return x + 1
}

func main() {
      var a, b, c int64
      fmt.Print("Masukkan nilai x : ")
      fmt.Scan(&a)
      fmt.Print("Masukkan nilai y : ")
      fmt.Scan(&b)
      fmt.Print("Masukkan nilai z : ")
      fmt.Scan(&c)
      result1 := f(g(h(a)))
      result2 := g(h(f(b)))
      result3 := h(f(g(c)))
```

```
            fmt.Printf("F(G(H(%d))) : %d\n", a, result1)
            fmt.Printf("G(H(F(%d))) : %d\n", b, result2)
            fmt.Printf("H(F(G(%d))) : %d\n", c, result3)
}
```

Output :



Penjelasan :

Kode program ini mengimplementasikan konsep fungsi komposisi matematika $(f o g o h)(x) ke dalam bentuk kode pemrograman. Terdapat tiga fungsi mandiri yang dibuat, yaitu fungsi f(x) untuk menghitung nilai kuadrat, g(x) untuk melakukan pengurangan dengan angka 2, dan h(x) untuk melakukan penambahan dengan angka 1. Pada bagian utama program, pengguna diminta memasukkan tiga nilai input untuk variabel x, y, dan z. Program kemudian melakukan pemanggilan fungsi secara bersarang (nested), di mana hasil dari satu fungsi langsung diteruskan sebagai input untuk fungsi berikutnya sesuai urutan komposisi yang diminta. Terakhir, hasil dari setiap komposisi fungsi tersebut ditampilkan ke layar.

3. Soal Unguided 3

1. Skiena dan Revilla dalam Programming Challenges mendefinisikan sebuah deret bilangan. Deret dimulai dengan sebuah bilangan bulat n. Jika bilangan n saat itu genap, maka suku berikutnya adalah ½n, tetapi jika ganjil maka suku berikutnya bernilai 3n+1. Rumus yang sama digunakan terus menerus untuk mencari suku berikutnya. Deret berakhir ketika suku terakhir bernilai 1. Sebagai contoh jika dimulai dengan n = 22, maka deret bilangan yang diperoleh adalah:
   22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
   Untuk suku awal sampai dengan 1000000, diketahui deret selalu mencapai suku dengan nilai 1.
   Buatlah program **skiena** yang akan mencetak setiap suku dari deret yang dijelaskan di atas untuk nilai suku awal yang diberikan. Pencetakan deret harus dibuat dalam prosedur cetakDeret yang mempunyai 1 parameter formal, yaitu nilai dari suku awal.
   `procedure cetakDeret(in n : integer)`
   Masukan berupa satu bilangan integer positif yang lebih kecil dari 1000000.
   Keluaran terdiri dari satu baris saja. Setiap suku dari deret tersebut dicetak dalam baris yang dipisahkan oleh sebuah spasi.

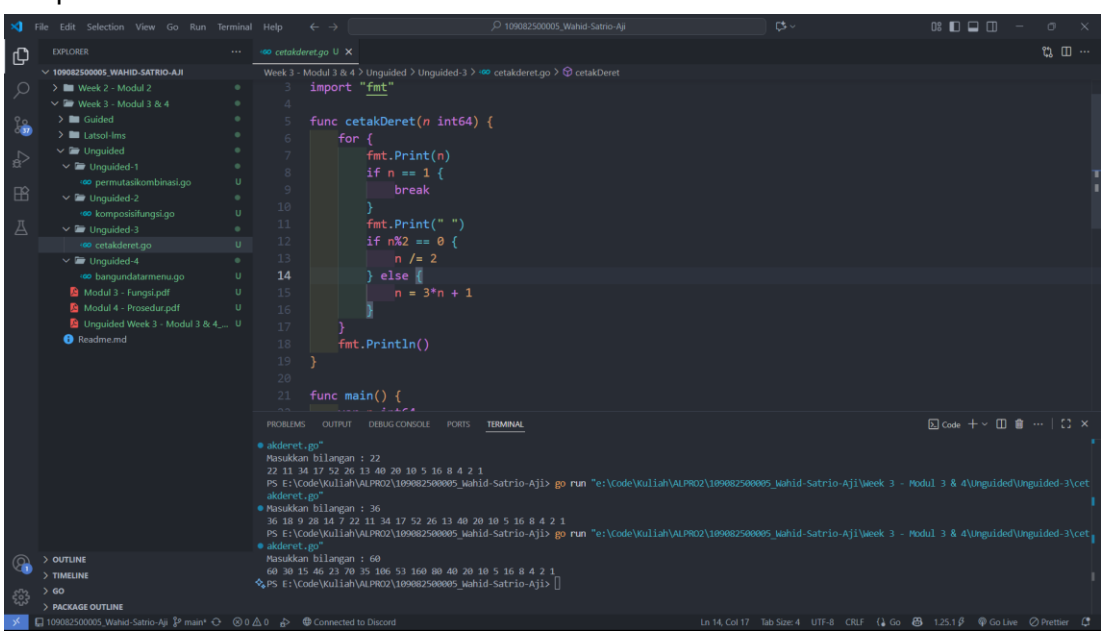
No	Masukan	Keluaran
1	22	22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
2	36	36 18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
3	60	60 30 15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1

Jawaban :

```
package main

import "fmt"

func cetakDeret(n int64) {
```

```
            for {
                  fmt.Print(n)
                  if n == 1 {
                        break
                  }
                  fmt.Print(" ")
                  if n%2 == 0 {
                        n /= 2
                  } else {
                        n = 3*n + 1
                  }
            }
            fmt.Println()
      }

      func main() {
            var n int64

            fmt.Print("Masukkan bilangan : ")
            fmt.Scan(&n)

            cetakDeret(n)
}
```

Output :



Penjelasan :

Program ini menggunakan sebuah prosedur bernama cetakDeret untuk menghasilkan urutan angka berdasarkan aturan tertentu hingga mencapai angka 1. Logika yang diterapkan di dalam prosedur ini adalah menggunakan

perulangan tak terbatas (for) yang berisi kondisi pengecekan: jika angka saat ini bernilai genap, maka angka tersebut dibagi dua (n/2), namun jika bernilai ganjil, maka angka tersebut dikalikan tiga lalu ditambah satu (3n+1). Setiap perubahan angka akan langsung dicetak dalam satu baris. Perulangan baru akan berhenti menggunakan perintah break tepat saat nilai angka tersebut sudah menyentuh angka 1. Program ini memudahkan pengguna untuk melihat alur perubahan angka dari input awal hingga mencapai titik akhir deret secara otomatis.

4. Soal Unguided 4

2. Buatlah sebuah program yang dapat menghitung luas & keliling persegi, persegi panjang, dan lingkaran. Setiap operasi perhitungan bangun datar dibuat prosedurnya sendiri-sendiri :

```
Procedure hitungPersegi(sisi : integer)
{I.S. terdefinisi bilangan bulat sisi
 F.S. menampilkan hasil perhitungan luas & keliling persegi}

Procedure hitungPersegiPanjang(panjang, lebar : integer)
{I.S. terdefinisi bilangan bulan Panjang & lebar
 F.S. menampilkan hasil perhitungan luas & keliling persegi
Panjang}

Procedure hitungLingkaran(jarijari : float64)
{I.S. terdifinisi bilangan decimal jarijari
 F.S. menampilkan hasil perhitungan luas & keliling lingkaran}
```

Buat menu tampilan awal program seperti berikut ini. (hint : gunakan percabangan switch-case)

```
--- PROGRAM BANGUN DATAR ---
1. Hitung luas & keliling persegi
2. Hitung luas & keliling persegi panjang
3. Hitung luas & keliling lingkaran
Pilihan :
```

Kemudian hitunglah luas & keliling :

1) Persegi dengan sisi 1057
2) Persegi Panjang dengan Panjang 106 dan lebar 88
3) Lingkaran dengan jari-jari 13,87

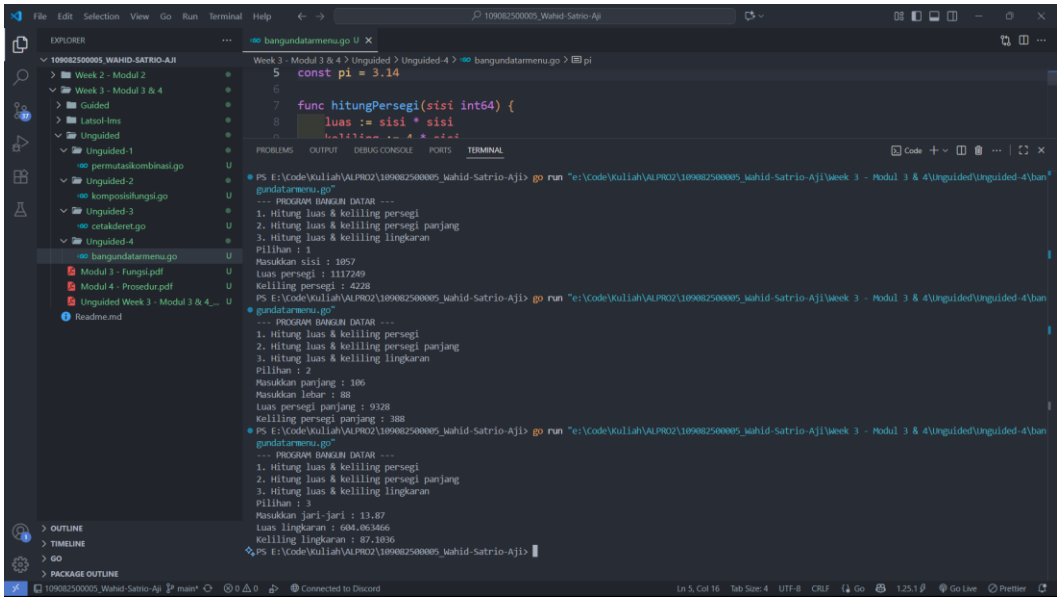
Jawaban :

```
package main

import "fmt"
```

```
const pi = 3.14

func hitungPersegi(sisi int64) {
	luas := sisi * sisi
	keliling := 4 * sisi
	fmt.Printf("Luas persegi : %d\n", luas)
	fmt.Printf("Keliling persegi : %d\n", keliling)
}

func hitungPersegiPanjang(panjang, lebar int64) {
	luas := panjang * lebar
	keliling := 2 * (panjang + lebar)
	fmt.Printf("Luas persegi panjang : %d\n", luas)
	fmt.Printf("Keliling persegi panjang : %d\n",
keliling)
}

func hitungLingkaran(jarijari float64) {
	luas := pi * jarijari * jarijari
	keliling := 2 * pi * jarijari
	fmt.Printf("Luas lingkaran : %.6f\n", luas)
	fmt.Printf("Keliling lingkaran : %.4f\n",
keliling)
}

func main() {
	fmt.Println("--- PROGRAM BANGUN DATAR ---")
	fmt.Println("1. Hitung luas & keliling persegi")
	fmt.Println("2. Hitung luas & keliling persegi
panjang")
	fmt.Println("3. Hitung luas & keliling
lingkaran")
	fmt.Print("Pilihan : ")

	var pilihan int
	fmt.Scan(&pilihan)

	switch pilihan {
	case 1:
		var sisi int64
		fmt.Print("Masukkan sisi : ")
		fmt.Scan(&sisi)
		hitungPersegi(sisi)
	case 2:
		var panjang, lebar int64
		fmt.Print("Masukkan panjang : ")
		fmt.Scan(&panjang)
		fmt.Print("Masukkan lebar : ")
		fmt.Scan(&lebar)
		hitungPersegiPanjang(panjang, lebar)
	case 3:
```

```
                var jarijari float64
                fmt.Print("Masukkan jari-jari : ")
                fmt.Scan(&jarijari)
                hitungLingkaran(jarijari)
            default:
                fmt.Println("Pilihan tidak valid")
            }
}
```

Output :



Penjelasan :
Program ini merupakan aplikasi kalkulator sederhana yang dirancang untuk menghitung luas dan keliling dari tiga jenis bangun datar, yaitu persegi, persegi panjang, dan lingkaran. Secara struktur, program ini menerapkan konsep prosedur dalam bahasa Go, di mana setiap rumus perhitungan dipisahkan ke dalam blok fungsi tersendiri agar kode lebih modular dan mudah dibaca.

Pada bagian awal, program mendefinisikan konstanta pi dengan nilai 3.14 yang akan digunakan khusus untuk perhitungan lingkaran. Terdapat tiga prosedur utama:

a. hitungPersegi: Menerima input berupa sisi, lalu menghitung luas (sisi × sisi) dan keliling (4 × sisi).

b. hitungPersegiPanjang: Menerima input panjang dan lebar, kemudian menghitung luas dan keliling dengan rumus standar.
c. hitungLingkaran: Menerima input jari-jari bertipe float64 untuk mengakomodasi angka desimal, lalu menghitung luas dan keliling menggunakan konstanta pi.

Di dalam fungsi utama atau main, program menampilkan menu kepada pengguna. Setelah pengguna memasukkan angka pilihan (1, 2, atau 3), program menggunakan struktur switch-case untuk menentukan alur selanjutnya. Misalnya, jika pengguna memilih angka 2, program akan meminta input panjang dan lebar, lalu memanggil prosedur hitungPersegiPanjang untuk memproses data tersebut. Jika input yang dimasukkan tidak sesuai dengan menu yang tersedia, program akan menjalankan bagian default yang menampilkan pesan bahwa pilihan tidak valid.

Secara keseluruhan, alur logika program ini dimulai dari menampilkan menu, menerima pilihan user, meminta input nilai dimensi bangun datar yang dipilih, melakukan kalkulasi melalui prosedur terkait, dan terakhir menampilkan hasil perhitungannya secara langsung ke layar. Penggunaan prosedur di sini sangat membantu agar fungsi main tidak terlalu penuh dan logika setiap perhitungan menjadi lebih terorganisir.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan pada Modul 3 dan 4 ini, dapat disimpulkan bahwa penerapan fungsi dan prosedur sangat krusial dalam menciptakan program yang terorganisir. Dengan membagi tugas ke dalam bagian-bagian kecil atau modular, kode program menjadi jauh lebih mudah dipahami dan proses pencarian kesalahan (debugging) menjadi lebih cepat karena setiap bagian memiliki tanggung jawab yang jelas. Kita tidak perlu lagi menulis kode yang sama berulang kali, cukup memanggil fungsi atau prosedur yang sudah dibuat.

Selain itu, pemahaman mendalam mengenai penggunaan pointer atau pass by reference memberikan fleksibilitas lebih dalam mengelola memori dan data. Kita jadi tahu kapan harus mengirimkan salinan data agar lebih aman, dan kapan harus mengirimkan alamat memori asli agar data bisa diubah secara langsung. Secara keseluruhan, penguasaan materi subprogram ini menjadi modal yang sangat kuat

untuk membangun logika pemrograman yang lebih kompleks namun tetap terjaga kerapiannya. Teknik ini membantu kita beralih dari sekadar "menulis kode" menjadi "merancang program" yang profesional dan sistematis.